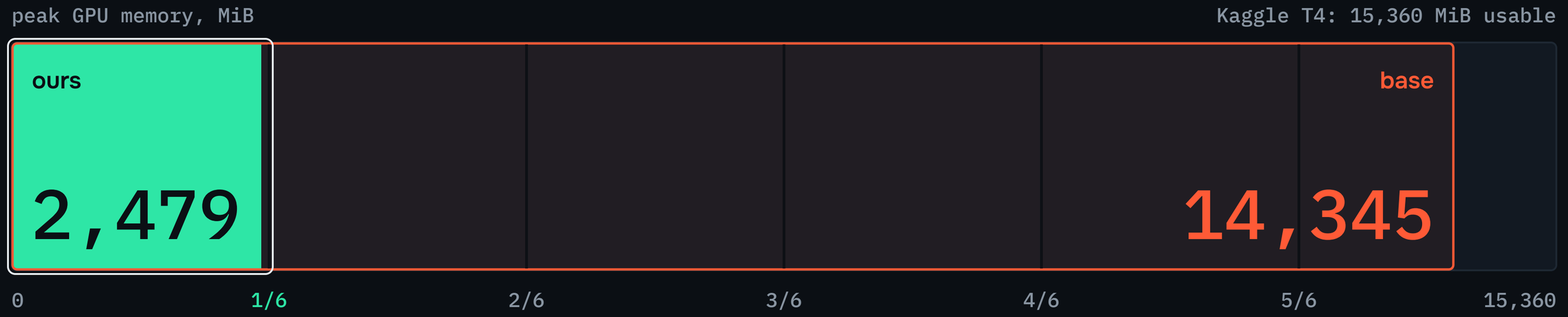
KaskyrAlmaty Soup Kernel Challenge, Decentrathon 6.0

One small fine-tune fills a free GPU. We made it fit in a sixth.

Same data, same steps, same eval loss. Peak memory measured by nvidia-smi.



Memory decides who can fine-tune at all.

The GPU most people can get is **a free 16 GB Kaggle T4**.



a ready model
28 layers, frozen

your data

your model
LoRA adapters learn

2,479 MiB the same fine-tune with our kernels

12,881 MiB free: a cheaper card will do, or other work runs alongside



We may change how it computes, not what it computes.

Locked going in

- model — Qwen2.5-1.5B, NF4
- data — 256 rows, same order
- batches — 4 × 512, 64 steps
- LoRA — r 16, α 32, dropout 0.05
- optimizer — AdamW, cosine

How it computes

our own Triton kernels

what stays in memory

what is rebuilt, and when

ours to change; its time counts

Locked coming out

- the same LoRA adapter
- loss at every step
- entropy, token accuracy
- rewind rows
- held-out eval loss

10 admission checks

1 2 3 4 5 6 7 8 9 10

all 10 pass

$$\text{score} = 35 \cdot \ln(s')/\ln(1.5) + 30 \cdot \ln(m')/\ln(3)$$

s, m = base ÷ ours, for time and memory; s′, m′ = 1.0 when the ratio is under 1.05.

one fails: not admitted, 0 points

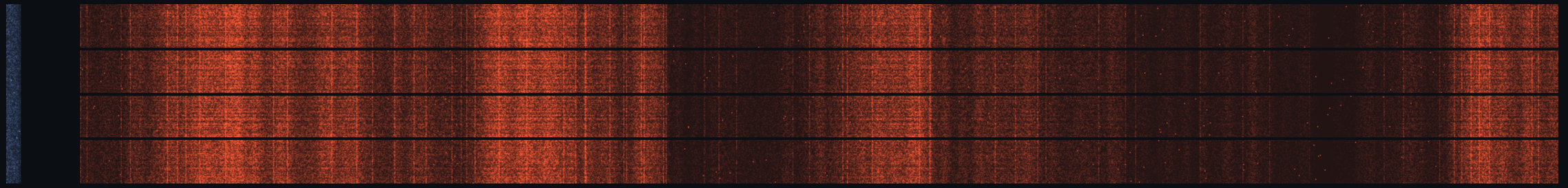
14,345 MiB

15,360 MiB T4



Two things fill it, and neither needs to be stored.

1,536 each token's 1,536 numbers become 151,936 scores, one per vocabulary token 151,936



$$4 \times 512 \times 151{,}936 = 311{,}164{,}928 \text{ numbers}$$

× 4 bytes in fp32 = 1.24 GB for one copy

1 pixel here ≈ 1,000 numbers

Copies of it, est. 5–7 GB

loss gradient entropy accuracy rewind

28 layers, activations kept for backward

forward → ← backward



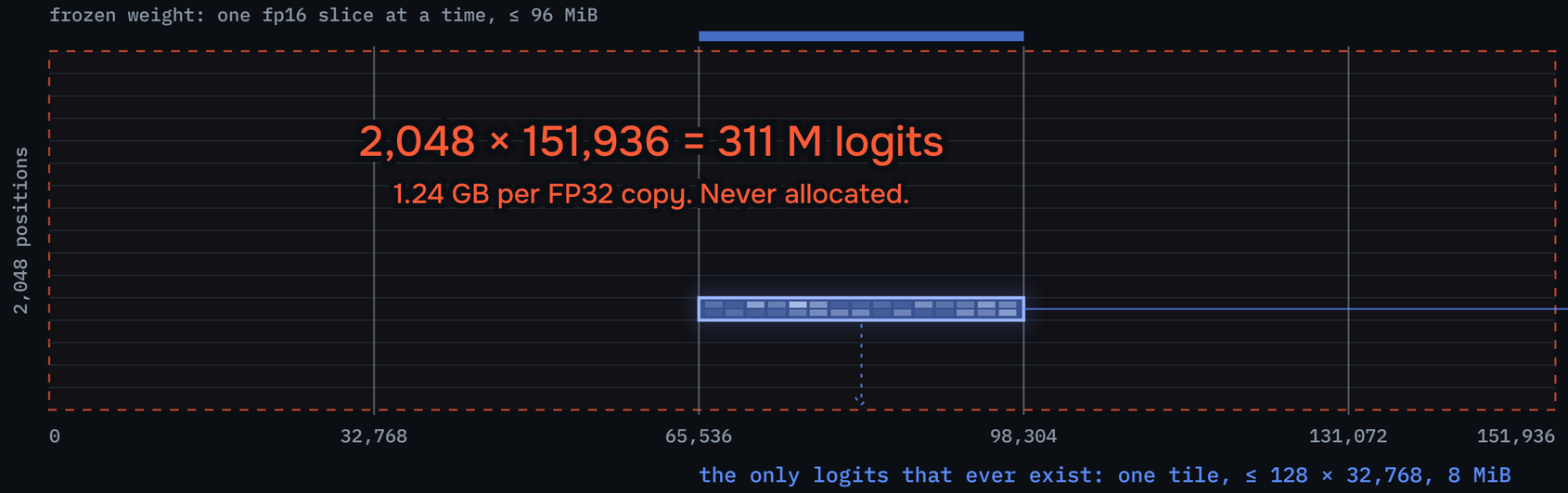
14,345 MiB 15,360 MiB T4



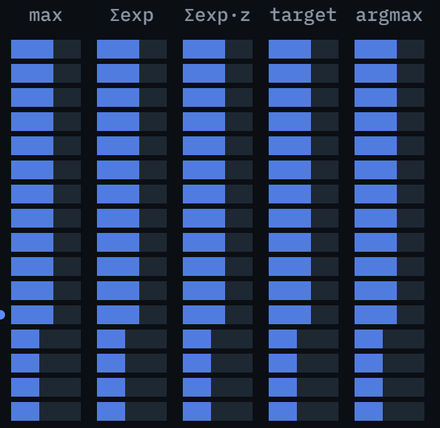
Cut 1: the 151,936-wide logits never exist.

Seven own Triton kernels. Ideas from Liger fused linear CE and Apple Cut Cross-Entropy; the code is ours.

Forward: one tile of logits at a time, folded into running numbers



5 numbers per position



bar: share of the vocabulary folded in

```
m' = max(m, max z)
s' = s·e^(m−m') + Σ e^(z−m')
```

`head_stats_partial`, `head_online_merge`
`head_stats_finish` → `loss, entropy, accuracy`

Backward: the same sweep, each gradient value split into two fp16 halves

$$0.001234568 = 0.001235 - 7{,}564 \,/\, 2^{34}$$

FP32 gradient — fp16 high — fp16 low, scaled

fp16 alone is off by 0.04%; the pair is exact in FP32

$$hi \cdot W + lo \cdot W \,/\, S$$

two cuBLAS products, summed in FP32

`head_residual_max`, `head_residual_scale`,
`head_ce_compensated`, `head_accumulate_compensated`

vs independent FP64 autograd

`rtol 0.01 atol 0.003`

✓ passes

5,283 MiB — 14,345 → 5,283 MiB: streamed head on top of the first body recompute (head alone: 9,661 → 5,283)

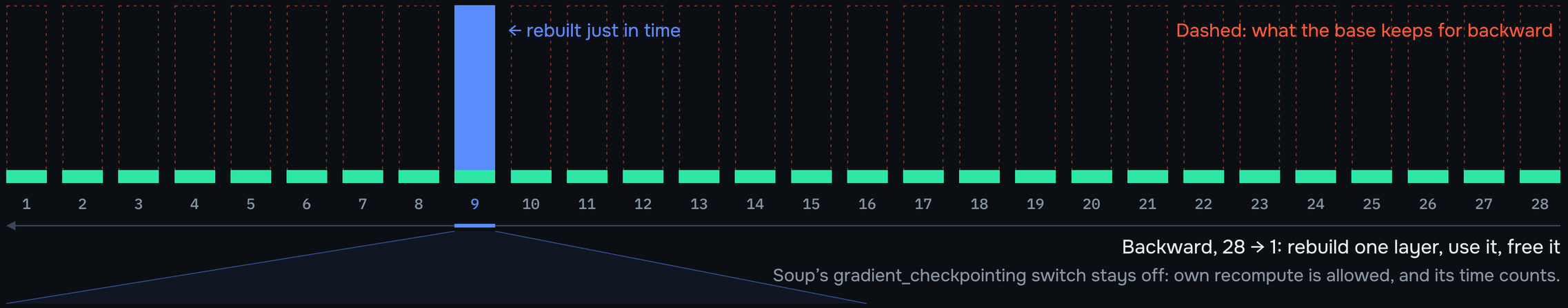
15,360 MiB T4



Cut 2: forget a layer's activations, rebuild them exactly.

Body lane, Timur Ishmurzin, `solution/body/`

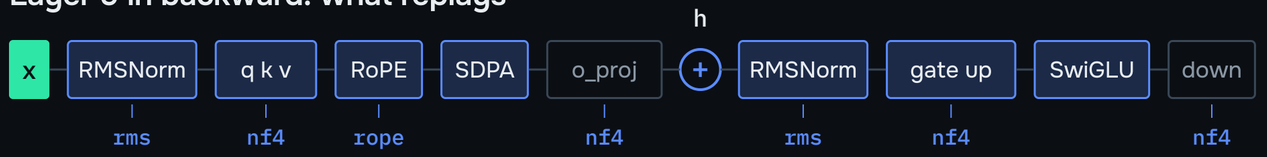
Forward, layers 1 → 28: each layer keeps only its input



Backward, 28 → 1: rebuild one layer, use it, free it

Soup's gradient_checkpointing switch stays off: own recompute is allowed, and its time counts.

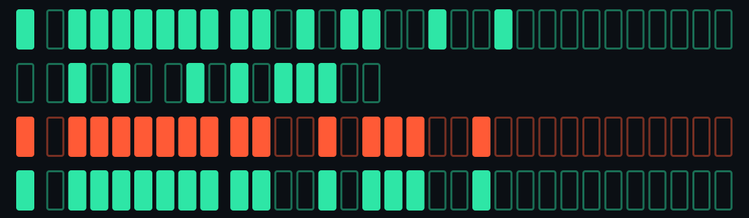
Layer 9 in backward: what replays



Own Triton kernels: `body_nf4_decode_weight` `body_rmsnorm_forward, _backward` `body_rope_token_kernel`

Blue ops replay with the RNG state restored: LoRA dropout 0.05 draws the same masks.

The FP32 residual sum, rebuilt bit for bit



Same operands, same FP32 add: 32 of 32 bits equal.

2,479 MiB — peak by nvidia-smi, Kaggle T4, run 1031-2312



We found it by measuring: ~550 ledger rows.

Two people, two AI orchestrators, one loop for 32 hours. Every dot is a run measured on a Kaggle T4.

- **Alikhan** + AI orchestrator
  head lane: the 151,936-token output layer
- **Timur** + AI orchestrator
  body lane: the 28 decoder layers

the orchestrator sets a hypothesis
an agent writes the kernel
a Kaggle T4 measures the peak
the result becomes a ledger row

- ▲ independent review every 3 h
- ◯ merge into main, reviewed by the other lane
- ○ killed idea, kept as a branch



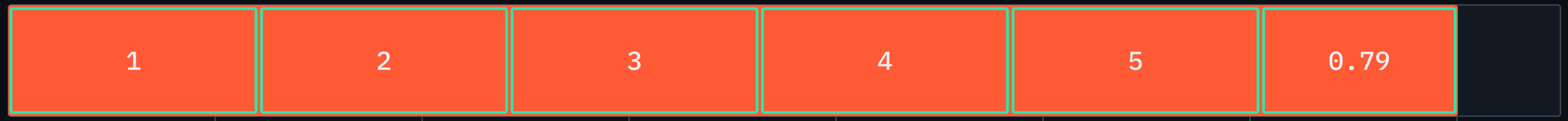


Same training, 5.8× less memory.

Peak GPU memory, read by nvidia-smi from outside the training process. Kaggle T4, run 1031-2312.

base 14,345 MiB

15,360 MiB usable on the T4



ours 2,479 MiB



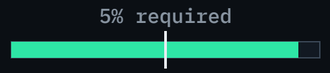
2.04967 → 2.04963

eval loss, base to ours: unchanged



10 of 10 admission checks pass

9.3%



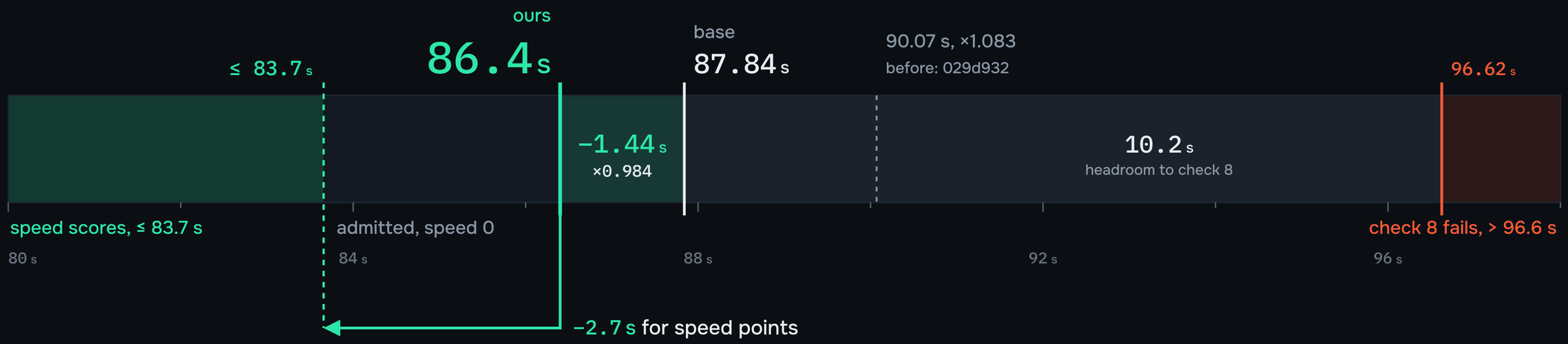
of training time in our own kernels

≈ 48

points from the memory term of the score

Faster than the base. 2.7 seconds from speed points.

Wall time of **`soup train`** on the Kaggle T4, run 1031-2312. 42.4% of token positions are padding; the MLP now runs only on valid rows. Before: **`029d932`** (2,541 MiB) ran ×1.083 with 1.37 s headroom; main before that failed check 8 at ×1.136.



Remove padding everywhere

through the whole model and the head, not only the MLP

Fused LoRA and cast kernels

fewer launches, no fp16 round trips through memory

Toward ~1 GB

fp16 tied-embedding storage and weight streaming (docs/ROADMAP-1GB-45S.md)



KaskyrAlmaty

14,345 → 2,479 MiB

Peak GPU memory of the same LoRA fine-tune on a Kaggle T4, run 1031-2312.

Alikhan Amirkhanov
captain, head lane

Timur Ishmurzin
body lane

code and ledgers	github.com/BAITC-Hacks/hack-bb68f34c-kaskyralmaty
notebook, run 1031-2312	kaggle.com/code/alikhanamirkhanov/kaskyr-head-2
this deck	aleka07.github.io/kaskyr-soup-deck

repository

